



DOCUMENTAÇÃO PARA DEFINIÇÃO DO FRAMEWORK E MoT

Sumário

1	Considerações Iniciais	3
2	Finalidade do Nó Sensor em uma Solução IoT.....	3
3	TpM	4
4	Framework e TpM	5
5	Framework e Modelo de Referência IoT.....	7
6	Tempo Real e Tempo Não real	9
7	Nível 4 é Interface entre Níveis 3, 5 e 6	10
7.1	Dados em tempo real:	11
7.2	Dados em tempo não real:	11
7.3	Exibição de gráficos.	12
7.4	Parâmetros.....	12
8	Coleta e Armazenamento de Dados em Tempo Real	12
9	MAC Distribuída e Centralizada	13
10	Tempo Real usando MoT	17
11	Pilha de Protocolo MoT	16
11.1	Camada de aplicação – APP	19
11.2	Camada de Transporte - TRANSP	19
11.3	Camada de Rede - NET	19
11.4	Camada de acesso ao meio – MAC.....	20
11.5	Camada física – PHY	20
12	Exemplo Framework com LoRa	21
12.1	Definição dos Níveis.....	21
12.2	Diagrama Temporal de fluxo de dados	22
13	Definição Kit-LoRa.....	24
13.1	Definição do Nó sensor – N1 e N2.....	24

13.2	Definição do Gateway N2 Camada Física	25
14	Protocolo MoT.....	Erro! Indicador não definido.
14.1	Pilha MoT	26
14.2	Comunicação Half Duplex	27
15	Pacotes MoT DL e UL	28
16	Exemplo Framework Básico	28
16.1	Acionamento do LED Amarelo	30
16.2	Python N2 e N3.....	30
16.3	Leitura do Comando do LED Amarelo	31
16.4	Pacote de DL	32
16.5	Pacote de UL.....	38
17	Organização Dados Brutos.....	42

1 Considerações Iniciais

O Framework é parte integrante da TpM para a implementação de uma solução IoT.

Esse documento estabelece as definições do **Framework básico** da TpM, criando assim um referencial para ser utilizado por todos os trabalhos no WissTek-IoT.

Não é objetivo aqui tratar da TpM em si, mas mostrar como o Framework é utilizado pela TpM.

O framework básico foi desenvolvido com o objetivo de apresentar um exemplo completo de solução IoT, incluindo todos os componentes necessários em cada nível do modelo de referência.

2 Finalidade do Nó Sensor em uma Solução IoT

O nó sensor em uma solução IoT apenas 2 funções em relação ao Nó Sensor sem fio:

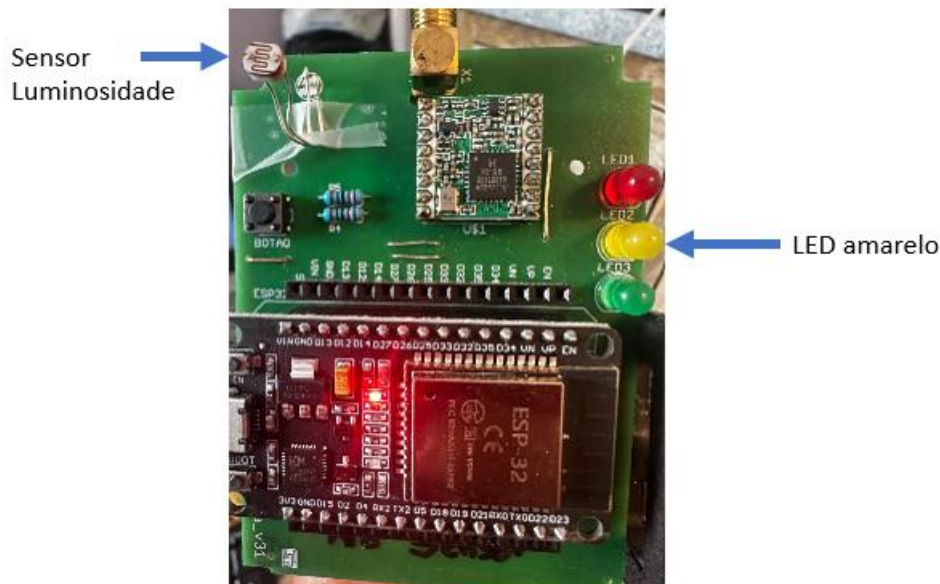
- Medir alguma grandeza através de um transdutor.
- Atuar para alterar algum status.

Para dar um exemplo do funcionamento do Framework foi criada uma solução IoT hipotética que tem esses dois propósitos:

- Medir alguma grandeza no local onde está o nó sensor, por exemplo a intensidade luminosa.
- Fazer algum comando no nó sensor, por exemplo acionar o LED amarelo.

Em resumo, uma solução IoT coleta dados no nó sensor, para serem analisados e eventualmente gera um comando para ser executado no nó sensor.

Esse material parte desse contexto utilizando para isso a PK-LoRa como nó sensor, que mede luminosidade e disponibiliza um LED amarelo para ser comandado.



Todas as questões envolvidas na medida de uma grandeza e para acionar algum processo no Nó Sensor serão tratadas nas seções seguintes, justificando cada ação.

3 TpM

Antes de entrar na parte prática é necessário entender em que contexto o Framework se enquadra.

Para criar a solução IoT foi criada no laboratório WissTek-IoT da Unicamp a TpM como definido no artigo do Carlinho.



Research article

The Three-Phase Methodology for IoT Project Development[☆]

Luiz Carlos B.C. Ferreira^{a,*}, Pedro R. Chaves^b, Raphael M. Assumpção^b, Omar C. Branquinho^c, Fabiano Fruett^b, Paulo Cardieri^a

^a Communications Department, Campinas State University, Campinas, Brazil

^b Semiconductors Instruments and Photonics Department, Campinas State University, Campinas, Brazil

^c PoB-Tec and the Extension Program of Computing Institute, Campinas State University, Campinas, Brazil



A Figura 1 desse artigo apresenta como a TpM é utilizada como metodologia para criar uma solução IoT.

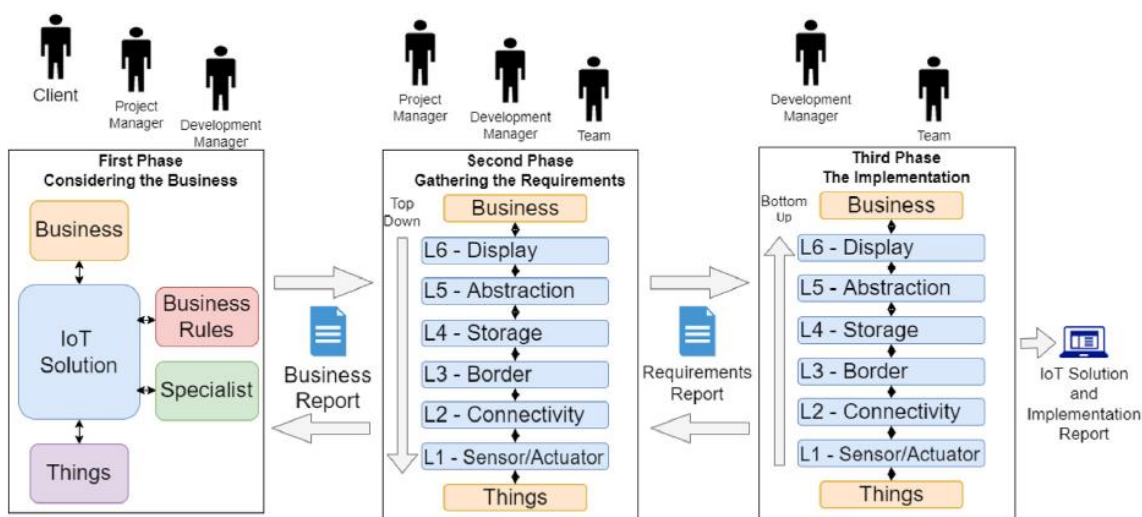


Fig. 1. TpM-Pro iterations diagram showing roles, phases and deliverables.

4 Framework e TpM

O artigo mostrado no item anterior apresenta a TpM como uma metodologia para orientar a criação de uma solução IoT.

Porém, não foi especificado como implementar efetivamente a solução IoT.

Para atender essa demanda foi criado o **Framework da TpM** com uma visão horizontal do modelo de referência para IoT.

O Framework foi proposto no artigo do Carlinho do SIoT de 2025.

A Systemic View on Implementing IoT Solutions

Luiz Ferreira
IFSULDEMINAS - Poços de Caldas
Poços de Caldas, Brazil
<https://orcid.org/0000-0003-4102-9742>

Pedro Chaves
Bosch Group
Campinas, Brazil
<https://orcid.org/0000-0003-4279-8850>

Raphael Assumpção
FEEC - Unicamp
Campinas, Brazil
<https://orcid.org/0000-0001-8438-642X>

Omar Branquinho
FEEC - Unicamp
Campinas, Brazil
<https://orcid.org/0000-0002-5193-8643>

Paulo Cardieri
FEEC - Unicamp
Campinas, Brazil
<https://orcid.org/0000-0002-7761-0240>

O Framework é uma visão sistêmica de uma solução IoT fazendo parte da Fase 3 da TpM.

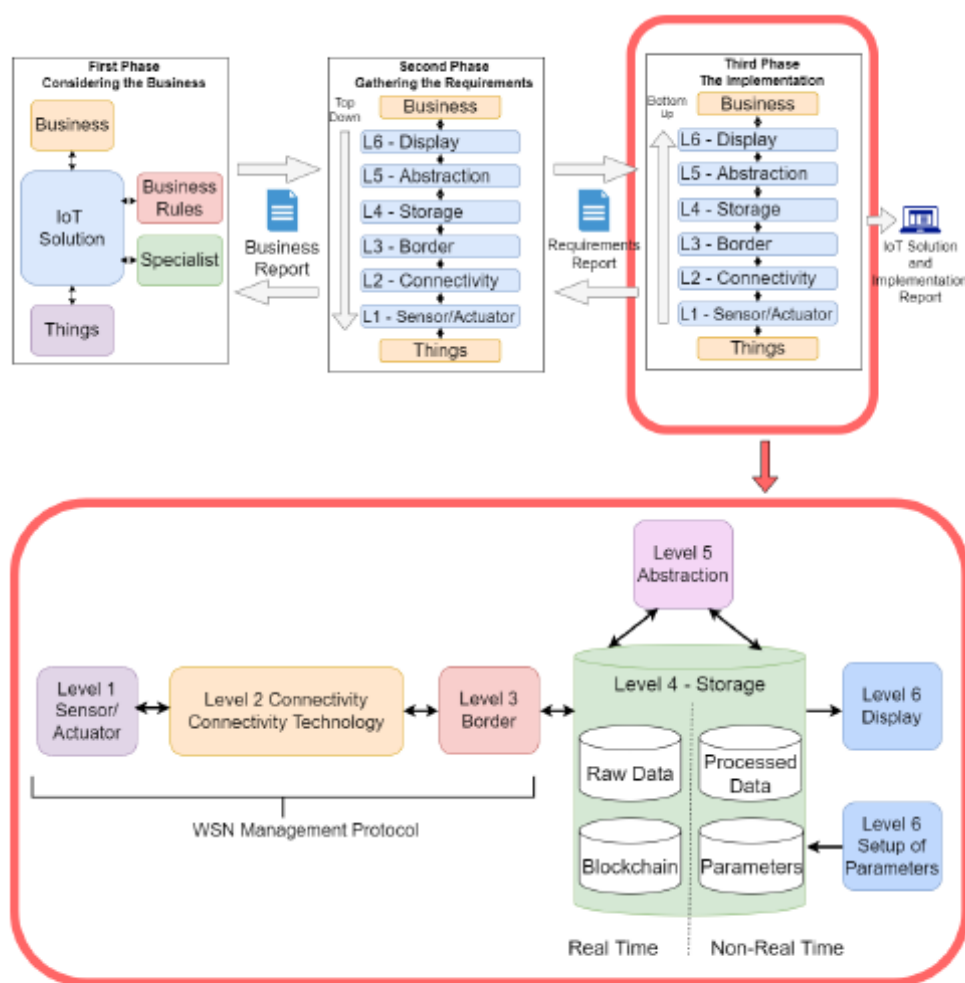


Fig. 4. TpM Framework

Como mostrado a figura que está na Figura 4 do artigo.

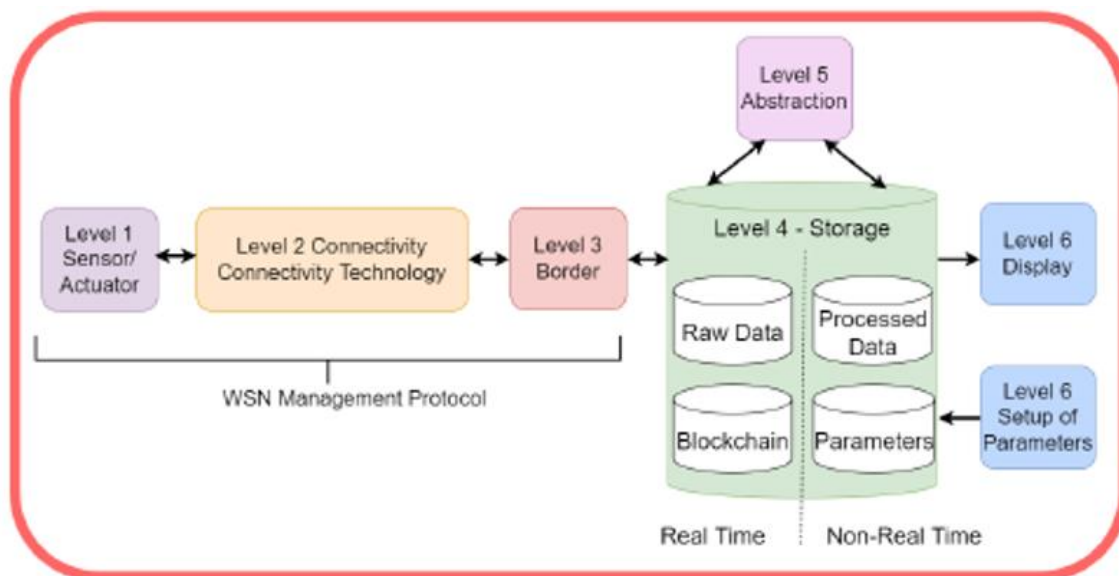
Para contextualizar a posição do Framework na TpM é necessário entender que ele somente é utilizado após as fases da TpM:

- Fase 1 – Negócio: análise do negócio, regras de negócio, áreas especialistas e definição das coisas.
- Fase 2 – Requisitos: utiliza o modelo de referência: exibição, abstração, armazenamento, borda, conectividade e coisas.
- Fase 3 – Implementação
 - 3.1 – Definição das tecnologias que utiliza a mesma visão vertical de baixo para cima: coisas, conectividade, borda, armazenamento, abstração e exibição
 - 3.2 – Implementação através do Framework que apresenta uma visão horizontal das partes do modelo de referência: processos locais, conectividade, borda, armazenamento, abstração e exibição.

Essa visão sistêmica tem por finalidade apresentar as partes da solução IoT indicando como será implementada.

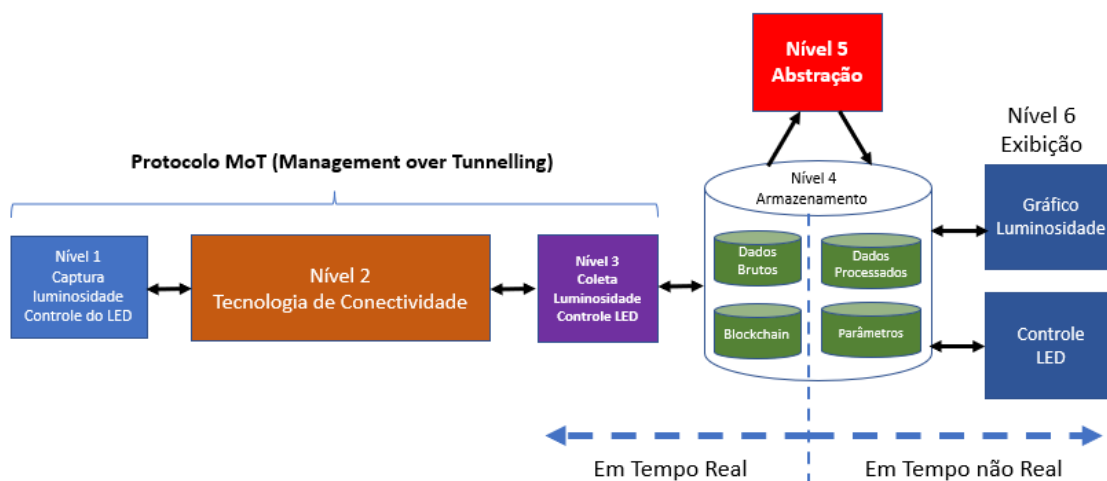
5 Framework e Modelo de Referência IoT

A implementação da solução IoT deve estabelecer como as tecnologias serão utilizadas para criar a solução, estabelecendo as questões de projeto de cada uma das partes.



O **Framework**, portanto, não identifica a tecnologia, mostrando apenas as funções que devem ser atendidas por cada um dos níveis.

Na próxima figura o Framework utiliza um código de cores para cada um dos níveis do modelo de referência, facilitando as explicações de cada nível.



Esse código de cores será especialmente importante para as definições necessárias para o Nível 2 – conectividade.

Analisando cada nível:

Nível 1 – faz a medida da luminosidade e comanda o LED amarelo.

Nível 2 – tecnologia de comunicação, que será o LoRa.

Nível 3 – coleta a luminosidade vinda do Nível 1 e envia comando para acionar LED no Nível 1.

Nível 4 – armazenamento de dados em tempo real e em tempo não real.

Nível 5 – processamento dos dados brutos para gerar os dados processados.

Nível 6 – exibição de gráfico da luminosidade e entrada de parâmetro para acionar o LED amarelo.

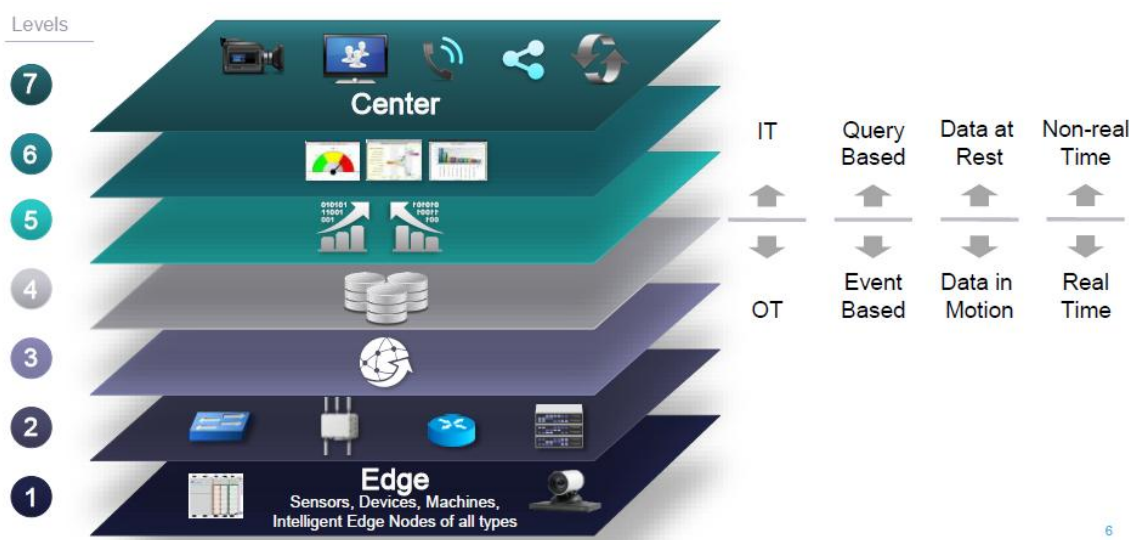
Essa divisão lógica das funções de cada nível é muito importante, pois permite identificar as tecnologias.

6 Tempo Real e Tempo Não real

Importante entender que o Nível 4 não especifica o tipo de armazenamento, mas os tipos de dados que são armazenados.

Um ponto fundamental é separar as partes que operam em tempo real, chamada de OT (Object Technology), e as partes que operam não em tempo real IT (Information Technology).

Essa definição foi utilizada da CISCO sendo muito apropriada para o Framework Básico, como mostrado na próxima figura.



Observar que essa divisão se dá no Nível 4 – armazenamento.

Analisando a figura do Framework Básico apresentada anteriormente, a parte em tempo real tem como coração da Solução IoT o Nível 3, que estabelece as ações de coleta de dados e execução de comandos e atuação no nó sensor.

A divisão entre tempo real e tempo não real é feita no Nível 4 – Armazenamento, que identifica as 4 funções de armazenamento:

- Tempo real

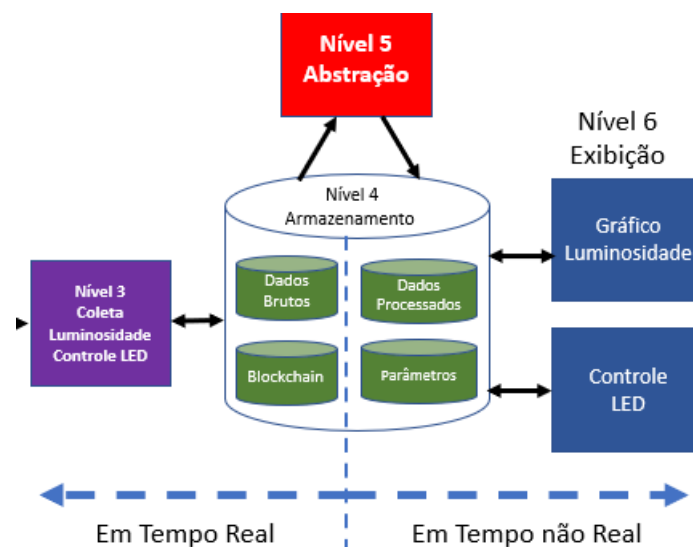
- Dados brutos são aqueles coletados em tempo real pelo Nível 3 e armazenados por ele no Nível 4.
- Blockchain é o armazenamento em tempo real através de blocos.
- Tempo não real
 - Dados processados pelo Nível 5.
 - Parâmetros para execução o Nível 1.

Para o estabelecimento da comunicação entre o Nível 3 e o Nível 1, através do Nível 2, foi criado o MoT (Management over Tunnelling).

Portanto, a atividade em tempo real depene de um protocolo de comunicação que é o MoT.

7 Nível 4 é Interface entre Níveis 3, 5 e 6

Para separar os dados que são coletados em tempo real e o dados em não tempo real o Framework utiliza o Nível 4 – Armazenamento – como a interface entre os níveis 3, 5 e 6.



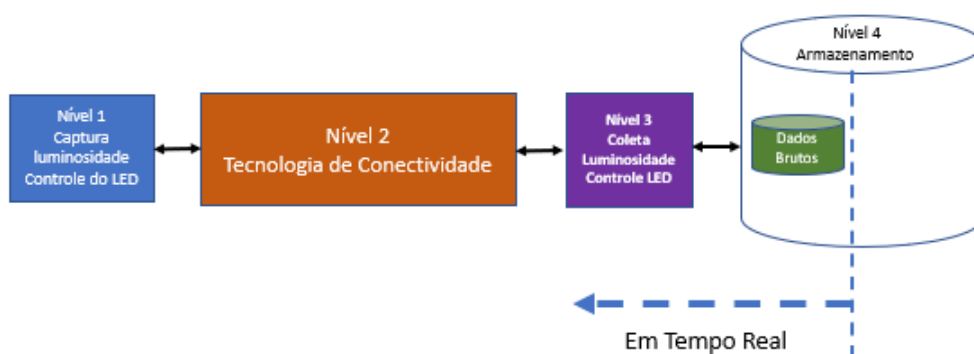
Esse é um ponto fundamental de entender que não existe a ligação direta entre os níveis 3, 5 e 6, sendo essa comunicação feita através de dados armazenados no Nível 4.

7.1 Dados em tempo real:

Dados brutos são os dados coletados pelo Nível 3 vindos do Nível 1, através de uma tecnologia de conectividade do Nível 2.

Os dados coletados são armazenados no nível 4 em tempo real.

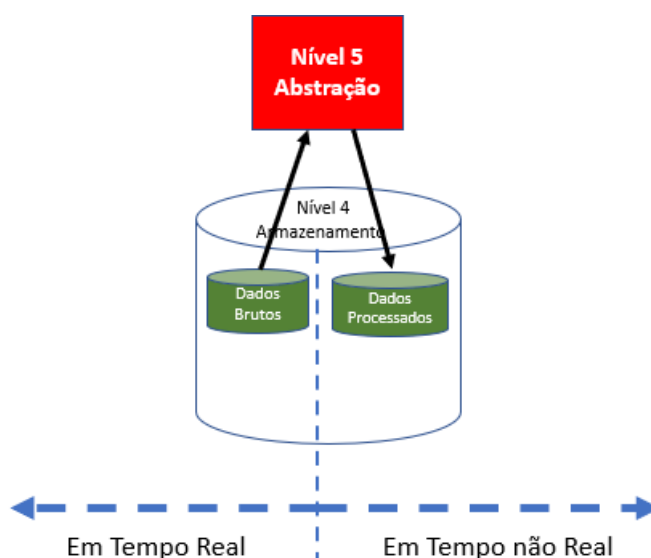
A comunicação entre o Nível 1 e 3 através do Nível 2 será detalhada mais a frente.



Blockchain – dados armazenados em tempo real garantindo integridade e segurança e não será tratado nesse documento.

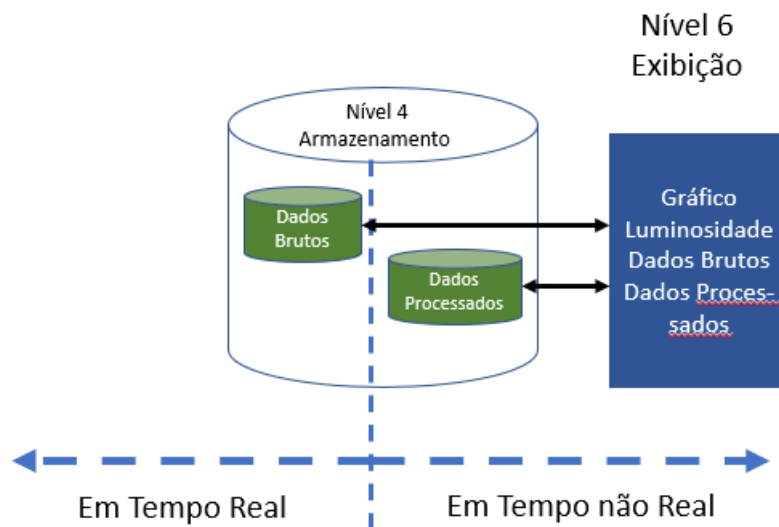
7.2 Dados em tempo não real:

Dados processados – dados oriundos do Nível 5 que processa os dados brutos.



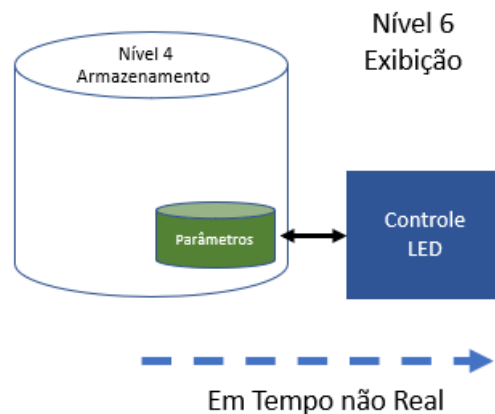
7.3 Exibição de gráficos.

Podem ser apresentados ao cliente os dados brutos que estão em tempo real e/ou os dados processados que estão em tempo não real.



7.4 Parâmetros

Armazenamento dos comandos para serem executados no Nível 1. No exemplo é comando do LED amarelo.



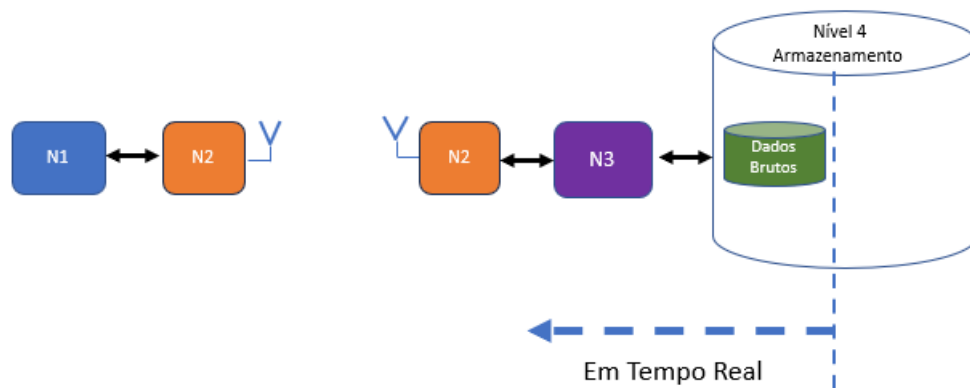
Níveis 3, 5 e 6 funcionam de forma assíncrona.

8 Coleta e Armazenamento de Dados em Tempo Real

A utilização de redes de sensores sem fio é uma necessidade em soluções IoT.

O Framework define como Nível 2 a tecnologia a ser usada para a comunicação sem fio, por exemplo LoRa.

Os dados são coletados através de uma Rede de Sensores Sem Fio, como mostrado na próxima figura.

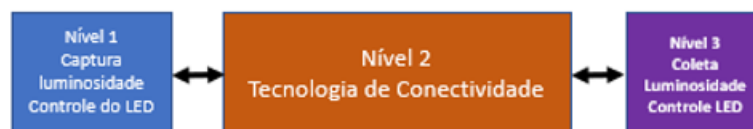


O Nível 3 tem como responsabilidade armazenar os dados brutos, como na figura.

Nessa figura está sendo considerado apenas um nó sensor, entretanto normalmente podem existir vários nós sensores.

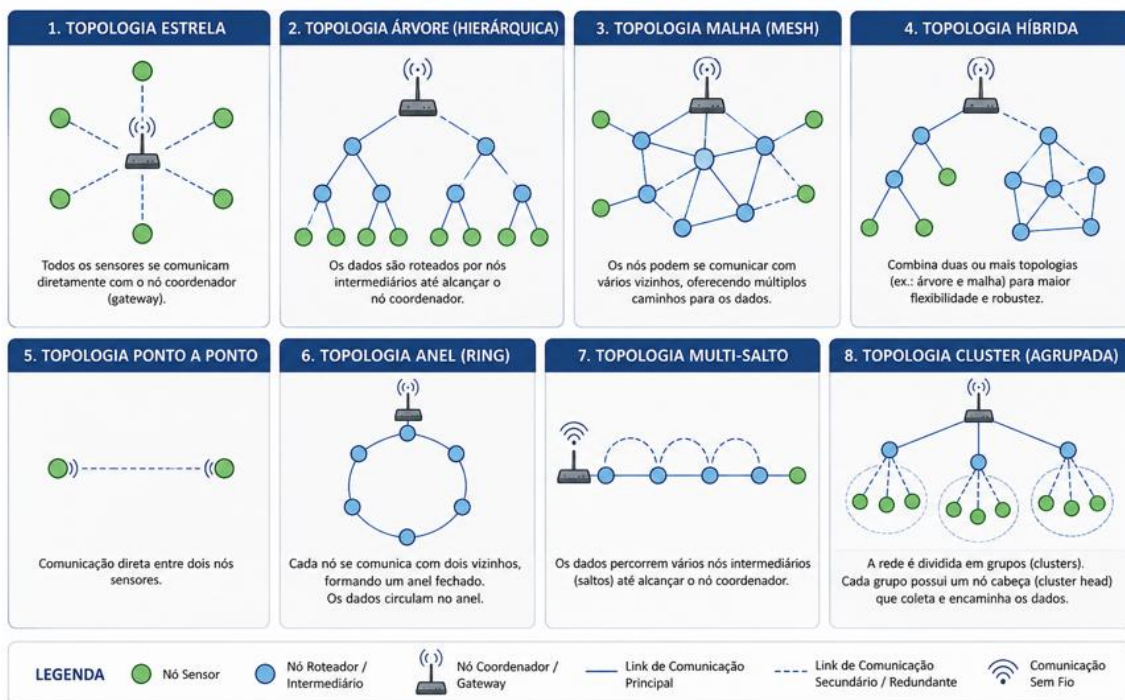
9 MAC Distribuída e Centralizada

O Framework faz a coleta de dados com a comunicação do Nível 1 com o Nível 3, através do Nível 2 que utiliza a tecnologia LoRa, por exemplo.



A organização da rede de sensores sem fio se dá através uma topologia da rede de dados.

TIPOS DE TOPOLOGIAS DE REDES DE SENSORES SEM FIO



Nessa material será utilizada a topologia estrela, com um gateway que se comunica de alguma forma com os nós sensores.

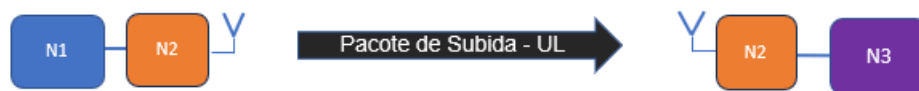


É preciso estabelecer a disciplina para a transferência das informações dos nós sensores para o gateway.

Basicamente é identificar qual elemento, gateway ou nó sensor, inicia a comunicação.

Para o Framework é definida a disciplina de comunicação como um controle de acesso ao meio MAC (Medium Access Control), que pode ser distribuído ou centralizado.

Na **MAC distribuída** a comunicação se inicia com o Nó sensor, ou seja, o Nível 1 gera um pacote e transmite para o Nível 3, denominado pacote de uplink UL, ou pacote de subida.



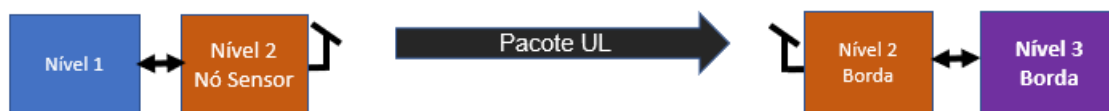
O problema dessa técnica distribuída é colisão de pacotes quando existe um número grande de sensores, com a necessidade de estratégias de mitigação de colisão.

Outra questão é quanto à estratégia de controle quando o Nível 3 deve enviar um comando para o Nível 1, por exemplo acionar o LED, necessitando uma coordenação entre os níveis.

Na **MAC centralizada** a iniciativa de comunicação se inicia no gateway, ou seja, o Nível 3 gera um pacote de solicitação, chamado pacote de downlink (DL) e envia para o Nível 1.



O Nível 1 responde com um pacote de uplink (UL) com a informação solicitada, no caso a luminosidade.



Nesse tipo de MAC não existem as duas questões mencionadas anteriormente, colisão e envio de comandos de controle, uma vez que o Nível 3 que envia um pacote e o Nível 1 responde.

Será utilizada a MAC centralizada para a implementação da solução IoT.

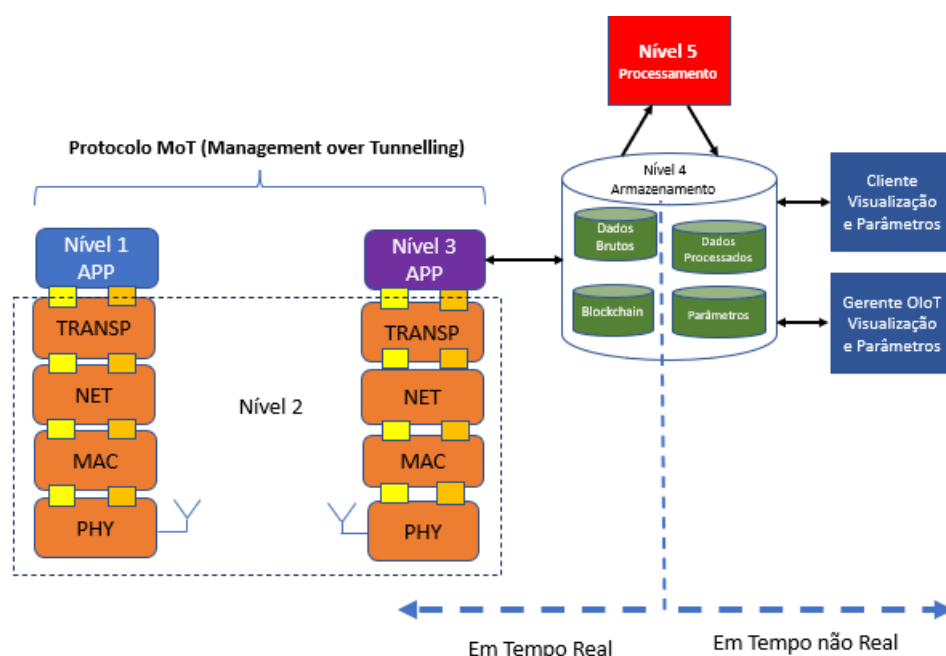
10 Pilha de Protocolo MoT

A definição de uma pilha de protocolos é fundamental para a comunicação entre os níveis 3 e 1.

É necessário definir responsabilidades para criar rede de comunicação de dados, a exemplo da Internet.

A pilha de protocolo MoT tem 5 camadas e na próxima figura considera a implementação do Kit-LoRa.

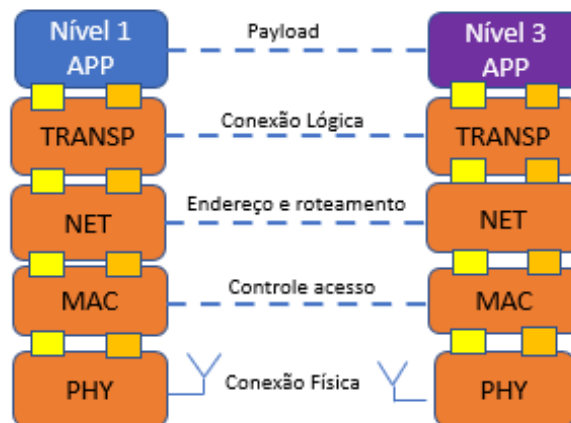
A próxima figura mostra a pilha de protocolo MoT e todas as parte do Framework TpM.



Entender a lógica de como é utilizada a pilha de protocolo é fundamental para a construção da RSSF gerenciada.

A seguir qual o objetivo de cada camada da pilha de protocolo.

Observar que as camadas estão ligadas uma a uma para gerenciar a RSSF.



A independência de cada par de camadas é fundamental para a evolução da pilha de protocolo.

Por exemplo, podem ser criadas estratégias para controle de taxa de sucesso de pacote na camada de transporte TRANSP, sem afetar as outras camadas.

Outro exemplo são estratégia de roteamento que estão na camada de rede NET.

A seguir uma descrição das principais funções de cada camada.

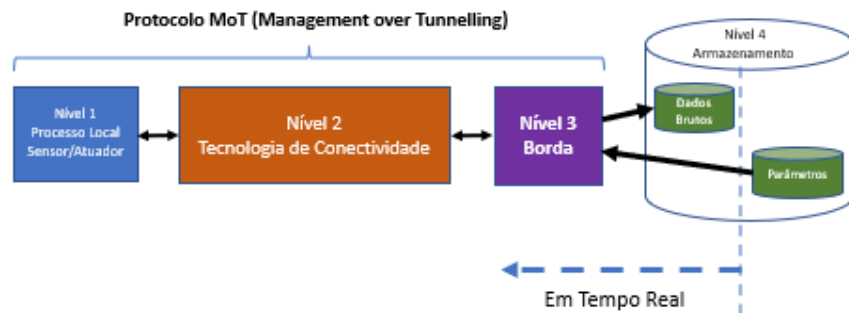
10.1 Tempo Real usando MoT

Para fazer a ligação entre o Nível 3 e o Nível 1, através do Nível 2, é definido para o Framework um protocolo de comunicação chamado MoT (Management over Tunnelling).

Para definição do MoT é necessário identificar as funções que o Nível 3:

- Medir alguma grandeza
- Atuar no Nó Sensor

A seguir as partes necessárias para essas duas funções.



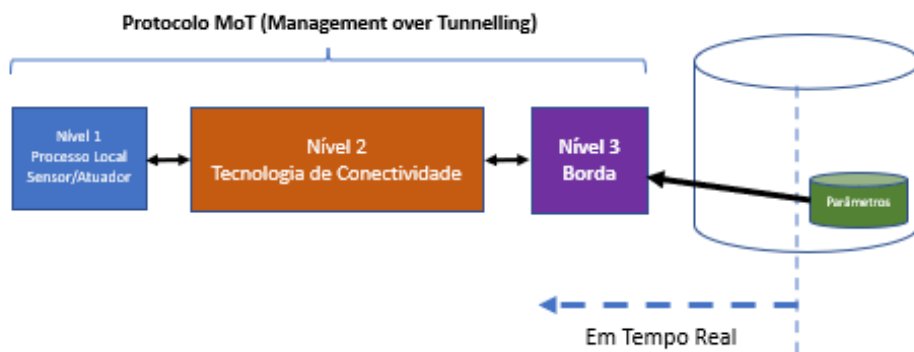
Para demonstração e simplicidade é utilizado um Python implementando o Nível 3.

Nível 4 serão dois arquivos TXT

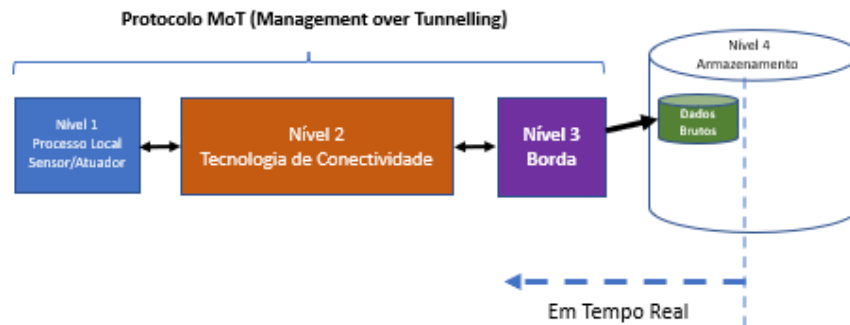
- Arquivo de parâmetro que tem a ação a ser executada no Nó Sensor
- Arquivo de dados brutos que terá a grandeza medida

O Python do Nível 3 interage com o Nível 4.

Primeiramente acessa o arquivo de parâmetros para executar algum comando no Nível 1.



Em seguida escreve no arquivo dados brutos do Nível 4 o dado coletado do Nível 1.



10.2 Camada de aplicação – APP

Camada de aplicação na borda N3 que vai enviar o comando para ligar o desligar o LED amarelo. Essa informação está no arquivo comando_led_amarelo.txt que é lido pelo Python para acionar ou não o LED amarelo.

- DL – Envio de comando para o Nó Sensor para comando do LED
- UL – Recebe luminosidade

Camada de aplicação no Nó Sensor N1

- DL – Recebe o comando de controle do LED
- UL – Medida de luminosidade e envia para gateway

10.3 Camada de Transporte - TRANSP

Camada de transporte faz o controle de fluxo dos pacotes entre N3 e n1, por exemplo:

- Contagem de pacotes enviados.
- Avaliação de pacotes perdidos.
- Reenvio de pacotes perdidos.

10.4 Camada de Rede - NET

Camada de rede para endereçamento e roteamento

- Cada elemento de uma rede de sensores sem fio deve ser identificado, por exemplo Sensor 1 e Base 0.
- Estratégia de roteamento para redes com múltiplos nós.

Exemplo de ações da camada de rede que é criar um híbrido LoRaWAN com LoRa.

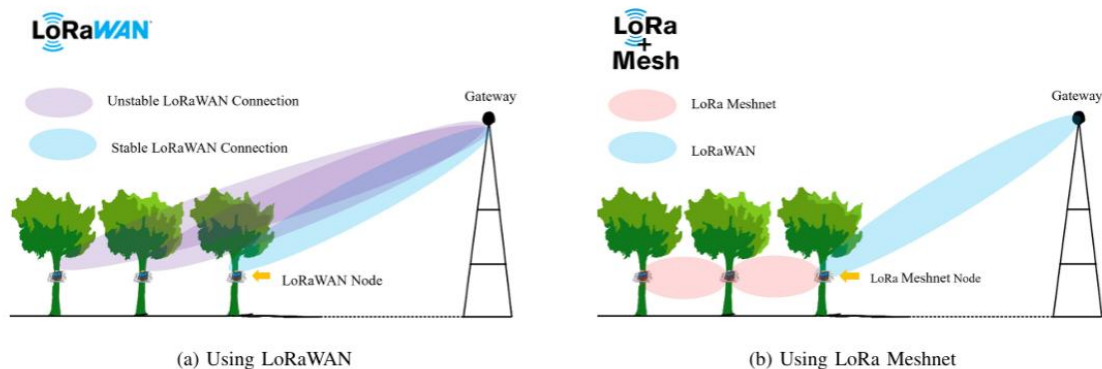


Fig. 1. The comparison of Fresnel zone between LoRaWAN and LoRa Meshnet.

10.5 Camada de acesso ao meio – MAC

- Tempo entre pacotes.
- Política de acesso ao meio.
- Gerência de energia do nó sensor

10.6 Camada física – PHY

Camada física com a definição da tecnologia de comunicação

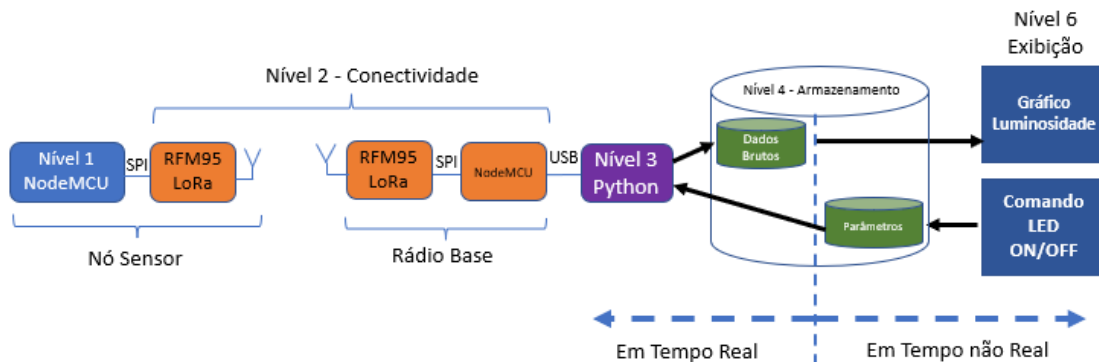
- LoRa
- LoRaWAN
- MQTT
- WiFi
- Bluetooth
- 4G
- USB

A camada física é a única que muda em função da tecnologia usada.

11 Exemplo Framework Basic com LoRa

A identificação dos níveis do modelo de referência para IoT utilizado na TpM é necessário entender como é feito o mapeamento com o Framework.

Para exemplificar a utilização da pilha de protocolo será utilizado o Kit-LoRa.



O “Motor” do framework é o Nível 3, que tem a dinâmica de operação em tempo real.

11.1 Definição dos Níveis

No exemplo apresentado na figura anterior existe uma sequência de ações que são desempenhadas pelos níveis do modelo de referência.

O N6 é a interface com o cliente e é dividido em duas partes:

- Exibição de gráfico da luminosidade e da média da luminosidade.
- Tela para opção de ligar o desligar o LED amarelo.

O N5 faz o processamento dos dados brutos e não será considerado nesse primeiro exemplo.

Basicamente o N5 processa os dados brutos armazenados pelo N3, que são exibidos no N6.

Por exemplo, a média da luminosidade que pode ser calculada a cada novo valor armazenado.

O N4 é tem uma função lógica de armazenamento e estabelece a interface entre os níveis 3, 5 e 6.

No exemplo serão usados 3 tipos de armazenamentos:

- Arquivo TXT com o status do LED, em função da opção do cliente.
- Arquivo TXT com os dados brutos da luminosidade coletada pelo N3. A cada nova medida é incluída no arquivo TXT.
- Arquivo TXT com a média da luminosidade, a cada novo valor da luminosidade é calculada a média de todos os valores anteriores e incluído no arquivo.

O Nível 3 é implementado em conjunto com a pilha de protocolo 2 através de um Python que implementa o N2 e N3 é o que faz a dinâmica da solução IoT.

O Nível 2 é a tecnologia de conectividade, que no exemplo é a interface LoRa

O Nível 1 trata dos processos locais, no exemplo o acionamento do LED amarelo e a medida da luminosidade.

11.2 Diagrama Temporal de fluxo de dados

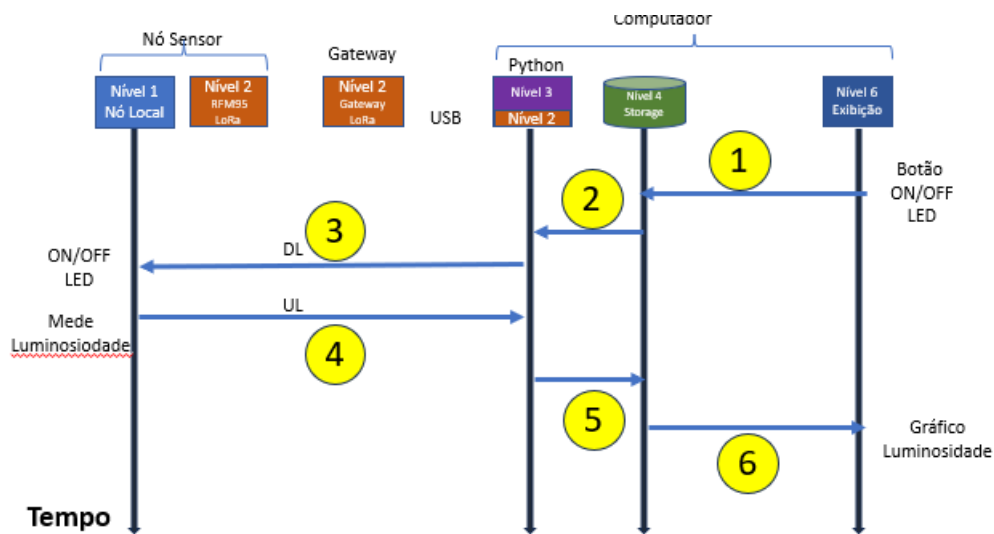
Para entender o fluxo dos dados a seguir está o diagrama temporal do fluxo dos dados.

Observar que os processos são assíncronos, o que significa dizer que acontecem de forma independente.

A seguir é mostrada a sequência de um acesso centralizado em que o N3 inicia a comunicação:

- DL: N6-N4-N3-N2-N1
- UL: N1-N2-N3-N4-N6

O Nível 5 será tratado a parte.



Fluxo dos dados assíncronos.

1 – Cliente tem um botão, implementado com Python, que pode ligar ou desligar o LED.

A opção do cliente é escrita em um arquivo no Nível 4 com valor 0 para LED apagado e 1 para LED aceso.

Nome do arquivo `cmd_led_amarelo.txt`

Ao clicar no botão salvar cria um arquivo de uma linha com valor 0 ou 1.

2 – Python implementa o Nível 3 e Nível 2. Isso é chamado de Borda lê o arquivo `cmd_led_amarelo.txt` e coloca no pacote byte 34 o valor 0 para desligar LED ou 1 para ligar o LED.

3 – Camada física do Python do Nível 2 envia pela USB para ESP32 que por sua vez envia para o módulo RFM95 via SPI.

4 – Nó sensor recebe o pacote DL pelo RFM95 que via SPI passa para camada física do EXP32.

5 – Pacote sobe a pilha MoT do nó sensor e na camada de aplicação e aciona LED se byte 16 for 1 ou apaga se for zero.

6 - Captura a luminosidade na camada de aplicação e coloca nos bytes 18 e 19 (inteiro e resto) e desce a pilha de protocolo e transmite o pacote de UL para o Gateway.

7 – Python Nível 3 recebe a luminosidade vinda o gateway Nível 2 e armazena em um arquivo chamado luminosidade.txt que vai registrando em tempo real as luminosidades.

8 – Python Nível 6 exibe luminosidade e RSSI de DL e UL.

12 Definição Kit-LoRa – Sensor e Gateway

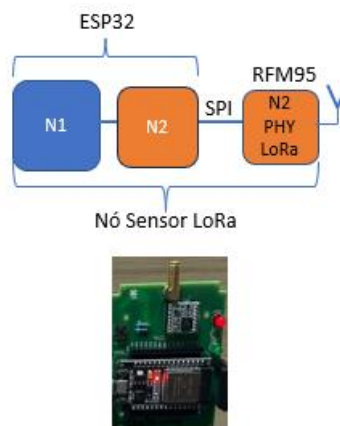
O Kit-LoRa é composto por um gateway e um nó sensor.

A seguir a definição das funções de cada um.

12.1 Definição do Nó sensor – N1 e N2

O nó sensor é composto de duas partes:

- ESP32
- Módulo RFM95



O ESP32 tem duas funções:

- Nível 1 que faz os processos locais para acionamento do LED e medida da luminosidade. É esse nível que tem contato com as coisas.
- Nível 2 conectividade que tem as funções necessárias para permitir a comunicação através da interface LoRa.

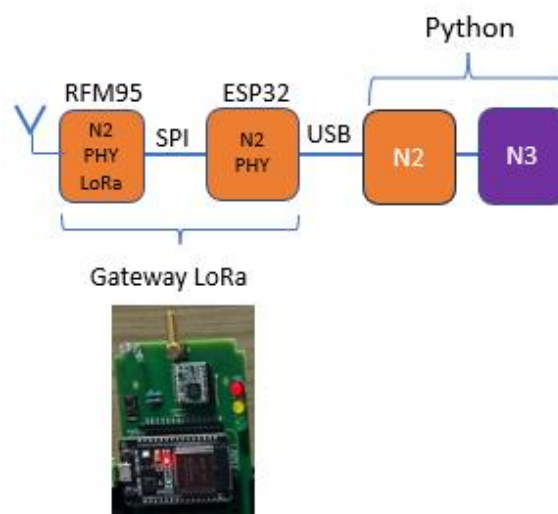
O ESP32 está ligado ao módulo RFM95 através do protocolo SPI.

Esse módulo tem a modulação LoRa, além de outras modulações.

12.2 Definição do Gateway N2 Camada Física

O gateway, diferentemente do nó sensor, é apenas a camada física que faz a interface entre o Nível 3 e a transmissão LoRa.

O ESP32 tem a função de ligar a USB do computador com o módulo RFM95 via SPI, e vice versa.



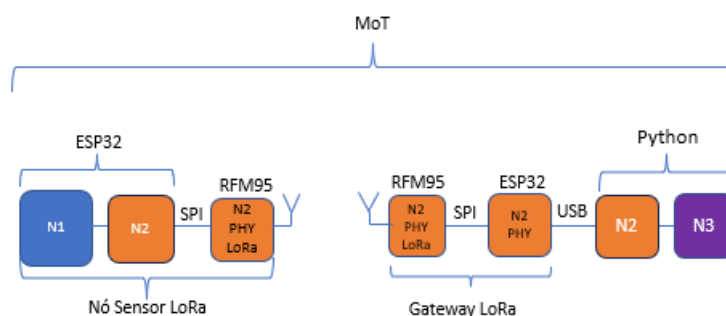
O Nível 2 no gateway tem, portanto, três partes:

- Python ligado via USB com o ESP32.
- ESP32 com código Arduino para ligação entre USB e SPI.
- Módulo RFM95 para comunicação com LoRa.

13 Níveis do Modelo de Referência para Kit-LoRa

As redes de comunicação de dados necessitam de uma pilha de protocolo, como acontece com o TCP/IP para a Internet, definindo responsabilidades bem específicas.

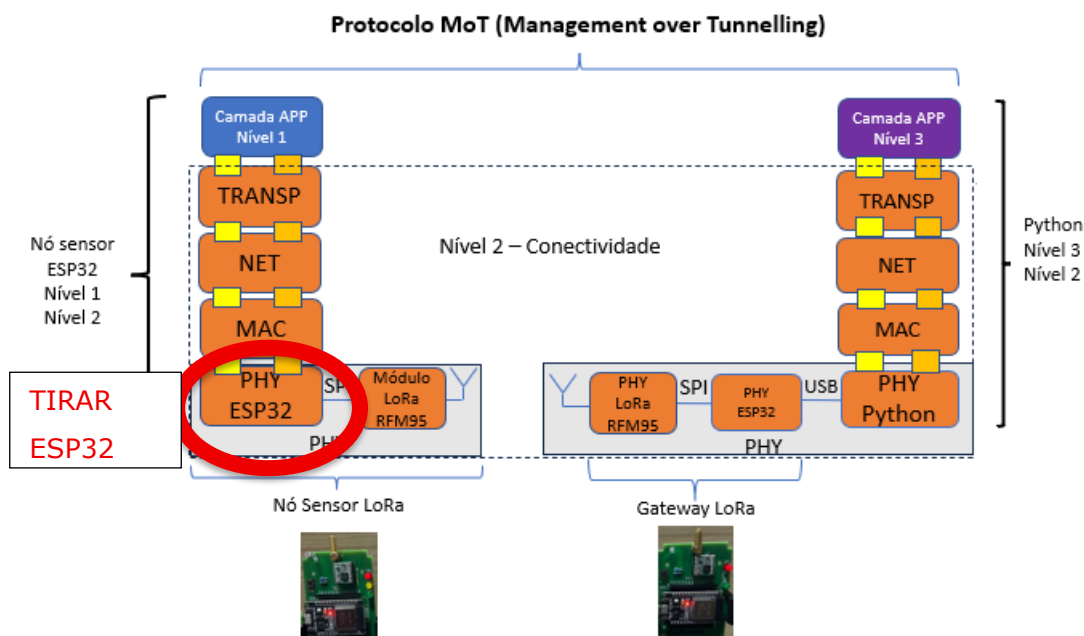
O protocolo MoT tem a função de ligar o Nível 3



O MoT é o protocolo de comunicação organizado em bytes para a comunicação entre o Nível 3 e Nível 1 e vice-versa.

14 Pilha MoT para Kit-LoRa

A próxima figura a pilha do protocolo MoT com suas 5 camadas já identificando o Kit-Lora com o nó sensor e o gateway.



O domínio da pilha de protocolo MoT é necessário para a criação de rede de sensores sem fio, que atendam aos requisitos de confiabilidade exigidos pelas soluções IoT.

O protocolo MoT é implementado com a criação de pacotes de down link e up link, que são compostos de bytes.

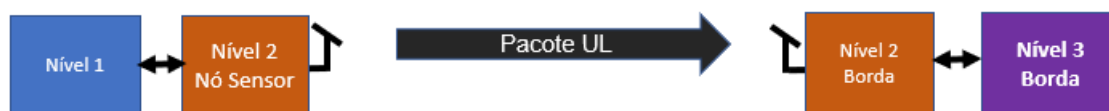
15 Comunicação Bidirecional Half Duplex

A comunicação na faixa de 915 MHz, em que opera o LoRa, é Half-duplex, ou seja, cada pacote ocupa tempos diferentes.

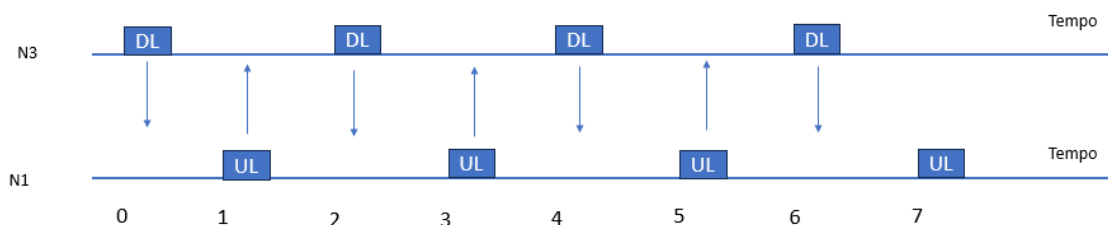
Como já mostrado, define-se pacote de DL aquele que vai do gateway N3 em direção ao nó sensor N1.



Define-se pacote de UL aquele que vai do nó sensor N1 em direção ao gateway N3.



No exemplo a seguir a iniciativa de comunicação parte do Nível 3 que envia um pacote de DL e o Nível 1 responde com um pacote de UL.



Define-se essa uma estratégia de acesso centralizada utilizado pelo exemplo.

16 Pacotes MoT DL e UL

Por questão didática o pacote é definido com o mesmo tamanho de 20 bytes de DL e UL.

Byte	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
Função	PHY				MAC				NET				TRANSP				APP			

Observar que o pacote pode ser bem menor que 20 bytes, mas por questões didáticas serão utilizados 20 bytes:

- Camada PHY – 4 bytes
- Camada MAC – 4 bytes
- Camada NET – 4 bytes
- Camada TRANSP – 4 bytes
- Camada APP – 4 bytes

O tamanho do pacote em uma Solução IoT vai depender das necessidades de medidas, controle e gerência.

Observar que o número do byte vai de 0 a 19, portanto 20 bytes.

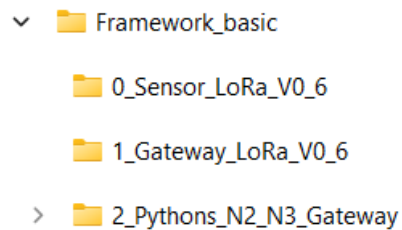
As informações que vão nos bytes de DL e UL são diferentes em função das necessidades para cada direção de comunicação.

Por questões didáticas foi mantido o mesmo tamanho dos pacotes de DL e UL.

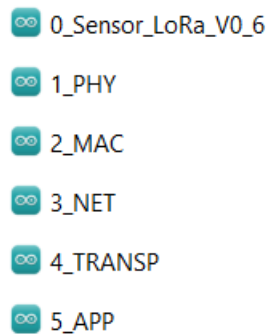
17 Exemplo Framework Básico

Nesse item será explicado o Framework Básico utilizado como referência, seguindo o diagrama temporal.

Os 3 subdiretórios do Framework Básico:



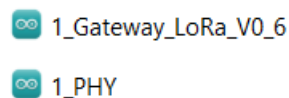
O primeiro subdiretório tem os códigos do ESP32 que vai no nó sensor.



Esse firmware implementa os níveis 1 e 2 que estão no nó sensor.

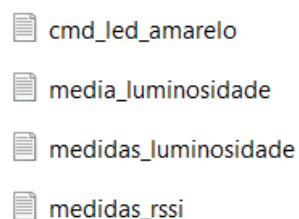
Os arquivos seguem a pilha de protocolo MoT.

Subdiretório do Gateway.



O Gateway tem apenas parte da camada física. Como já mencionado antes o ESP32 do gateway faz a ponte entre a USB do computador e a SPI do módulo RFM95.

Estrutura de arquivos do Nível 4

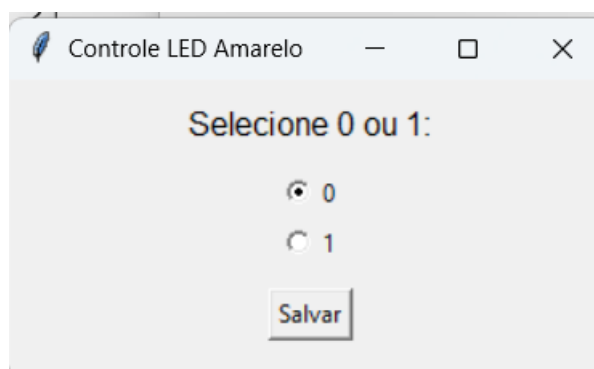


Pythons N3, N5 e N6.



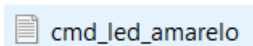
17.1 Acionamento do LED Amarelo

O Python que liga ou desliga o LED amarelo é bastante simples e tem a seguinte tela.



Esse Python cria um arquivo de comando do LED e dá opção para ligar com valor 1 e desligar com valor 0.

O arquivo criado no Nível 4 de armazenamento do status do LED:

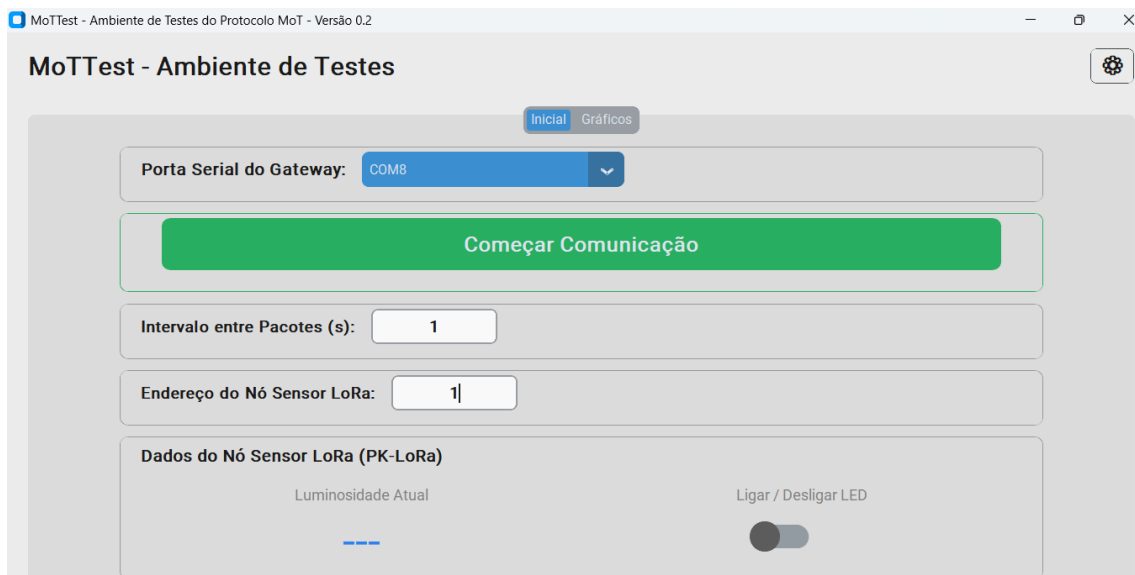


Esse é um arquivo de uma linha e uma coluna com o valor 0 ou 1, escrito quando clica em gravar.

17.2 Python N2 e N3

O Python que é o "Motor" do Framework tem por função gerar o pacote de DL e receber o pacote de UL.

Essa Python deve ser executado quando os firmwares do gateway e nó sensor já tiverem sido instalados e testados com o MoTTest.



E explicação sobre o teste do Kit-LoRa com o MoTTest está no documento de preparação do ambiente.

Importante, o MoTTest deve ser fechado para funcionar o Python N2_N3.

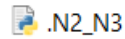
Ao executar esse Python é solicitada a COM do gateway e o número de medidas a serem feitas.

```
Digite o número da serial = 8
Entre com o número de medidas = 1000
```

Sequência de ações:

17.3 Leitura do Comando do LED Amarelo

O arquivo de comando do LED amarelo é lido no código do Python:



O trecho do código que faz essa leitura do arquivo do comando do LED amarelo.

```
# ===== Camada de aplicação PACOTE DL
# Lê o arquivo cmd_led_amarelo.txt
with open("cmd_led_amarelo.txt", "r") as f:
    linha = f.readline()
    # Remove espaços e ENTER
    linha = linha.strip()
    # Se o valor for 0 ou 1
    if linha == "0":
        Comando_LED_amarelo = 0
    elif linha == "1":
        Comando_LED_amarelo = 1
    else:
        # Qualquer outro conteúdo assume 0
        Comando_LED_amarelo = 0
except:
    # Se houver qualquer erro assume 0
    Comando_LED_amarelo = 0
# Coloca o comando no byte 16 do DL
Pacote_DL[16] = Comando_LED_amarelo
```

17.4 Pacote de DL

O pacote de DL leva informações de controle na camada de aplicação, que no caso é o comando do LED amarelo do Nó Sensor.



A seguir a definição das informações de cada byte do pacote de DL que está sendo utilizado pelo Kit-LoRa.

Pacote de Down Link - DL		
PHY	0	TBD
	1	TBD
	2	TBD
	3	TBD
MAC	4	Tempo entre pacotes
	5	TBD
	6	TBD
	7	TBD
NET	8	Identificação do sensor de destino
	9	Identificação do gateway de origem
	10	TBD
	11	TBD
TRANSP	12	Número de pacotes DL
	13	TBD
	14	TBD
	15	TBD
Dados DL	16	Comando LED amarelo
	17	TBD
	18	TBD
	19	TBD

A sigla TBD é para ser determinado no futuro.

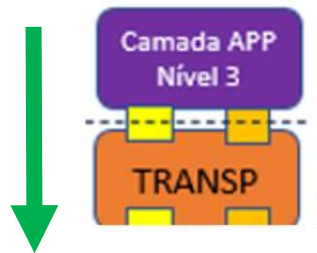
Será utilizado a seguir a sequência de down link completa:

- Python – Nível 3
- Firmware ESP2 gateway – Nível 2
- Firmware ESP32 nó sensor – Nível 2 e Nível 1

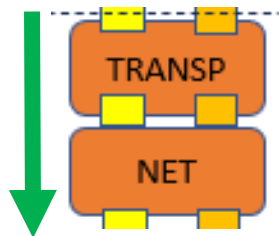
Camada APP DL – na camada de aplicação o pacote de DL envia o comando para ligar ou desligar o LED amarelo através do Byte 16.

Fica claro o desperdício na utilização de um byte para ligar e desligar um LED.

Nos desenvolvimentos é necessário utilizar o byte com mais eficiência levando mais informações.

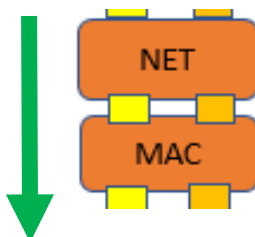


Camada TRANSP DL – informa no Byte 12 o número de pacotes já enviados pelo Nível 3. Nesse caso foi usado apenas um byte por simplicidade e no nó sensor é necessário tratar esse contador quando chega em 255.

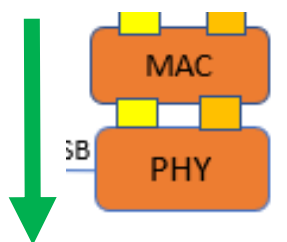


Camada NET DL – informa a identificação dos elementos de rede:

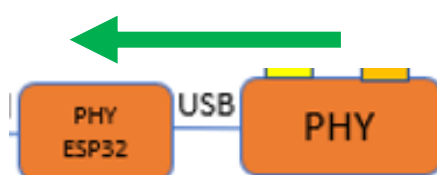
- Byte 8 – identificação do nó sensor que deve receber o pacote
- Byte 9 – identificação da base que originou o pacote de DL.



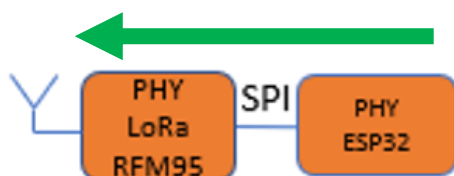
Camada MAC DL – informa no Byte 4 o intervalo entre os pacotes que serão transmitidos pelo Nível 3. Essa é uma informação importante para que o Nó sensor, eventualmente, entre em sleepmode, criando uma estratégia para economia de energia.



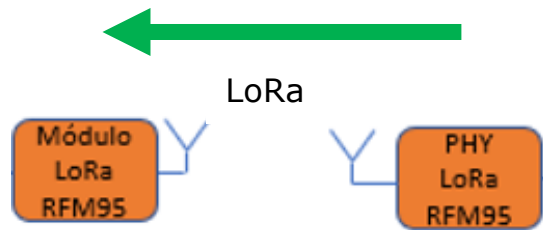
Camada PHY DL – Na evolução do protocolo MoT na camada PHY serão colocados os parâmetros do LoRa. O pacote é disponibilizado pela USB para ser lido pelo ESP32.



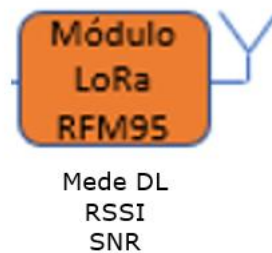
O ESP32 lê pela USB o pacote de 20 bytes e envia para o módulo RFM95 via SPI.



Após a transmissão pela interface LoRa é recebido pelo nó sensor.



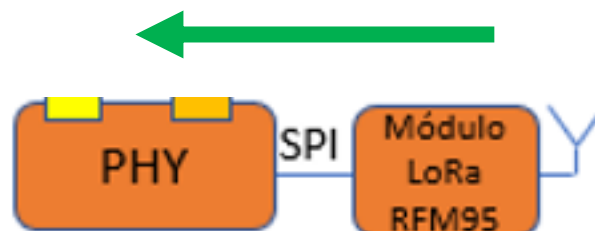
O RFM95 mede a RSSI DL e a SNR.



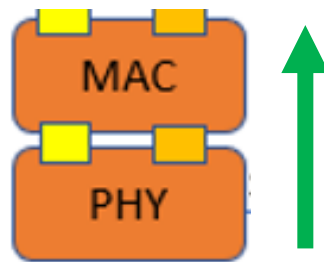
RFM95 disponibiliza dos bytes do pacote DL, RSSI e SNR para serem lidos pelo ESP32 pela SPI.

A leitura será feita pelo firmware do ESP32, como será explicado no código.

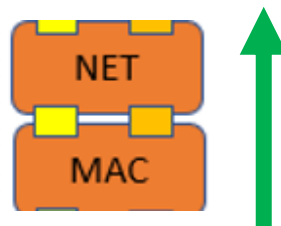
Camada PHY DL – leitura do pacote de DL, RSSI e SNR.



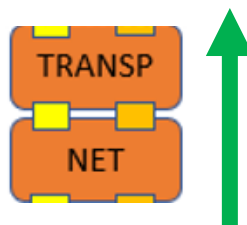
Camada MAC DL – leitura do tempo entre pacote no Byte 4 para sincronização.



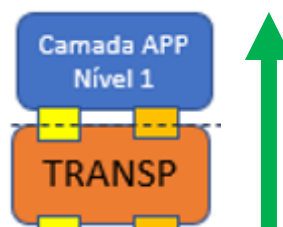
Camada NET DL – verifica a identificação no Byte 8 para checar se o pacote foi endereçado para esse nó sensor e enviado pelo gateway no Byte 9.



Camada TRANSP DL – camada de transporte recebe no Byte 12 o número de pacotes de DL e calcula se perdeu algum pacote de DL com parando com o byte anterior de DL.



Camada APP DL – a camada de aplicação executa o comando de apagar ou ascender o LED amarelo.



Interessante observar que a camada de aplicação da pilha de protocolo se confunde com o Nível 1 do modelo de referência.

17.5 Pacote de UL

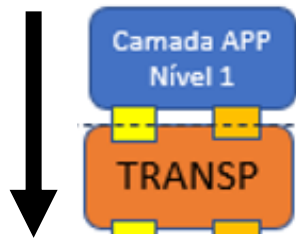
O pacote de UL é criado pelo Nível 1 e está dividido também em Header e Payload.



A seguir a definição das informações de cada byte do pacote de UL que está sendo utilizado pelo Kit-LoRa.

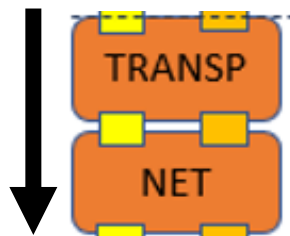
Pacote de Up Link - UL		
PHY	0	RSSI DL
	1	SNR DL
	2	RSSI UL
	3	SNR UL
MAC	4	Energia da bateria do nó sensor (Não implementado)
	5	TBD
	6	TBD
	7	TBD
NET	8	Identificação do gateway destino do pacote
	9	Identificação do nó sensor de origem
	10	TBD
	11	TBD
TRANSP	12	Número de pacote UL
	13	Perda de pacote DL
	14	TBD
	15	TBD
Dados UL	16	Status do LED
	17	Código do tipo de hardware PK-LoRa
	18	Inteiro luminosidade
	19	Resto luminosidade

Camada APP UL - Coloca o status do LED no Byte 16 de UL. Identifica através de um código qual o hardware e coloca no Byte 17, no caso a PK-LoRa. Faz medida da luminosidade e aloca inteiro no Byte 18 e resto no Byte 19.



Camada de TRANSP UL – a camada de transporte faz o controle de fluxo contabilizando o número de pacotes enviados e recebidos e contabilização de perdas de pacote.

- Byte 12 – número de pacote enviados de UL.
- Byte 13 – número de pacotes perdidos de DL.

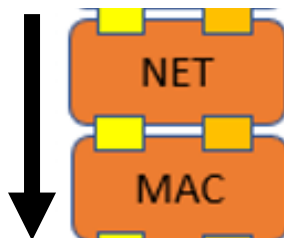


Camada NET UL – informa a identificação dos elementos de rede:

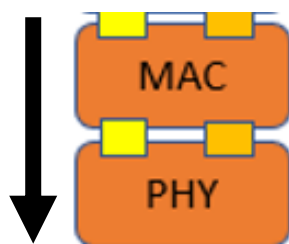
Byte 8 – identificação do gateway que deve receber o pacote de UL.

Byte 9 – identificação DO que originou o pacote de DL.

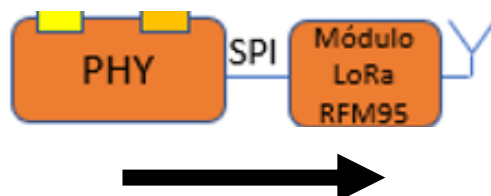
Em desenvolvimentos com múltiplos sensores é nessa camada que será criada a estratégia de roteamento.



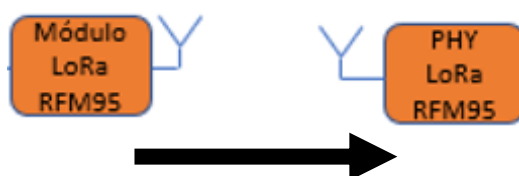
Camada MAC UL – informa no Byte 4 a energia das baterias do nó sensor. Na versão didática apresentada não está sendo gerada essa informação.



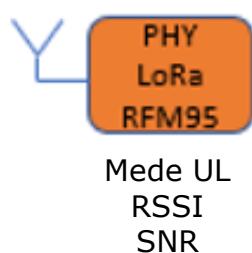
ESP32 envia pelo SPI o pacote para o RFM95.



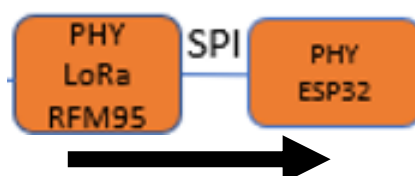
Pacote e transmitido pela interface LoRa



Camada PHY UL – Pacote é recebido pelo módulo RFM95 da borda. RFM95 mede RSSI UL e RSSI UL. Disponibiliza o pacote de UL para ser lido pelo ESP32.



Leitura pelo ESP32 via SPI do RFM95 do pacote UL com 20 bytes, RSSI UL e SNR UL.

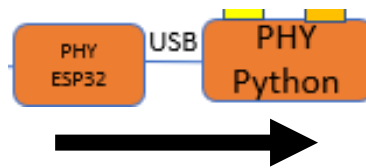


Escreve nos bytes

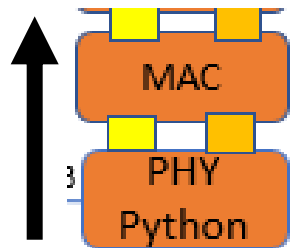
- Byte 2 – RSSI de UL medida no nó sensor
- Byte 3 – SNR de UL medida no nó sensor

Lembrando que nos bytes 0 e 1 estão RSSI DL e SNR DL, respectivamente.

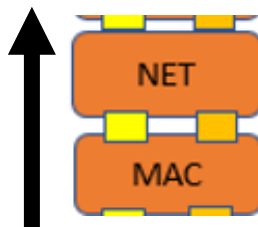
ESP32 disponibiliza o pacote com 20 bytes para ser lido pela camada PHY do Python.



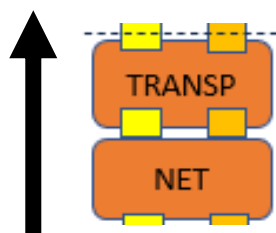
Camada MAC UL – recebe o nível de bateria no Byte 2.



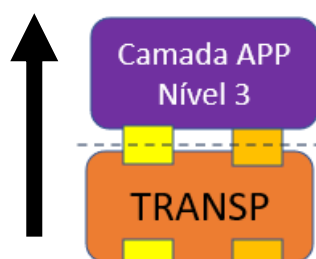
Camada NET UL – verifica a identificação do sensor que enviou o pacote no Byte 9 e verifica se o pacote é para essa borda no Byte 8.



Camada TRANSP UL – checa a contagem de pacotes de UL no Byte 12. Verifica o número de pacotes perdidos no DL Byte 13.



Camada APP UL – a camada de aplicação no UL tem por função organizar e armazenar os dados brutos coletados em tempo real no Nível 4.



18 Organização Dados Brutos

Esses dados são divididos em dois tipos:

- Dados para o cliente.
- Dados para gerência da RSSF.

